

How I Would Improve SmallFloats.jl

Improvement strategy

I would preserve the numerical architecture and improve the package from the outside inward. The projection and oracle modules already have substantial depth; rewriting them would create risk with little demonstrated benefit. The highest leverage work is to repair contradictions at public seams, automate the evidence the project already knows how to produce, and then optimize the allocation-heavy paths identified by the existing benchmark report.

The sequence below is ordered by risk reduction and user value, not by novelty.

1. Make public controls and reports truthful

Resolve the RNG and stochastic-bit defaults

First decide on one stochastic configuration model.

My preference is explicit entropy: scalar and array operations that use stochastic rounding should accept an RNG, while omitted RNGs use Julia's task-local `Random.default_rng()`. Projection precision should remain encoded in the projection specification. Under that design, deprecate and remove `DefaultRNG`, `SetDefaultRNG`, `DefaultRbits`, and `SetDefaultRbits` because process-global controls are unsafe in concurrent library code and currently provide no behavior.

If backward compatibility requires retaining them, then create one internal entropy/configuration seam and route all of these through it:

- `scalar_resolve_rng`;
- `rand` and `randn` convenience methods;
- stochastic projection-specification constructors;
- `block`, `array`, and generated operation wrappers.

Scoped overrides should be task-local, nest correctly, and restore state after exceptions. Tests should verify observed random-operation behavior, not merely getter/setter round trips. There must be no extra work in deterministic calls; typed dispatch should continue to let the compiler eliminate the RNG path.

Generate package identity once

Derive the version used by `conformance()` from package metadata or from one generated constant shared with `Project.toml`. Add the git commit and dirty-tree status when generating an external conformance artifact. The result should never contain a manually copied package version.

Record the P3109 draft with a precise document identifier and, if possible, a source hash. Maintain a short table of package interpretation decisions for underspecified draft behavior. This makes conformance reproducible rather than date-associated.

Unify the rounding taxonomy

Use the abstract rounding-mode hierarchy as the sole classification mechanism. Dispatch directed behavior on `DirectedRoundingMode`, faithful behavior on `FaithfulRoundingMode`, and so on. Delete the legacy unions, or mechanically derive compatibility aliases from the concrete subtype inventory.

Add an invariant check establishing that every exported concrete mode belongs to exactly the intended class. In particular, `ToOdd` should not simultaneously be faithful in the hierarchy and directed in a separate union.

2. Establish continuous, reproducible evidence

Add tiered CI

The repository already describes meaningful verification tiers. Turn those into automation:

1. Pull requests: formatting/static checks if adopted, Aqua, JET, documentation examples, and the package's default test tier.
2. Scheduled builds: exhaustive or release roll-call on supported platforms.
3. Releases: full conformance artifact, golden digests, documentation build, and package archive.

Use a small platform matrix that reflects actual support. Avoid pretending to support Julia versions that have not been evaluated. If Julia 1.12 is required, document the exact feature that requires it; otherwise investigate 1.11 compatibility as a separate, evidence-based task.

CI should publish machine-readable metadata: Julia version, target CPU, thread count, package commit, dirty state, test tier, assertion or case counts, and the exact P3109 draft identifier.

Make documentation derive from authoritative data

Replace duplicated facts with generated inclusions or consistency checks:

- package version;
- supported K range and format counts;
- exported rounding-mode taxonomy;
- number and names of session defaults;
- source-file references used by architecture pages;
- latest verification tier and case count;
- benchmark report date, commit, and selected headline measurements.

Rewrite test/README.md as an operational guide: tier definitions, expected duration, how sampling is selected, what golden files mean, and how to reproduce a failing gate. Move the historical review narrative into a dated design record if it remains useful.

Correct the current contradictions about sampling, assertion counts, removed files, default counts, and old timings. Expand the documentation consistency gate to check structured facts, not just a few literal strings.

Clean the repository and formalize releases

Stop tracking docs/pdf/build products. Keep the documentation source and, if desired, one named final PDF; publish other generated artifacts with releases or documentation builds. Add the build directory to .gitignore.

Create a release checklist covering version bump, changelog, draft identity, CI status, conformance artifact, documentation, tag, and benchmark-report metadata. Once the public interface is stabilized, tag releases and consider General registry submission.

3. Deepen and organize the public interface

Make representation conversion explicit

Introduce checked, intention-revealing functions such as:

```
codepoint(x)::Unsigned  
fromcodepoint(T, bits)
```

Keep an unchecked constructor internal, for example `_fromcodepoint_unchecked`, for kernels that have already established invariants. Deprecate `public rawvalue` if it bypasses validation, or redefine it as the clearly documented checked interface.

This removes the current semantic surprise where unsigned construction denotes a representation code point but signed construction denotes a numerical value. It also creates a stable

adapter between packed storage and the numeric type without exposing the codec implementation.

Reduce mutable implementation exports

Replace the exported `DEFAULT_SHOW_STYLE` Ref with `show_style()` and `set_show_style!()` or, preferably, an IO-context-based style mechanism. Public behavior should not depend on users mutating an implementation container.

Audit the export list and classify each name as one of:

- primary numerical interface;
- format catalog;
- advanced tuning/introspection;
- implementation detail.

Keep compatibility aliases where necessary, but organize discoverability through namespaces such as the existing `Formats` module and possible `Ops`, `Blocks`, or `Approx` modules. Do not replace the typed callable `Op` interface with symbols; the typed interface provides safer dispatch and better compiler leverage.

Document `Binary` as a closed dispatch/query type if external subtyping remains unsupported. Exporting an abstract type should not imply a supported extension seam that the implementation cannot honor.

Prefer explicit state in library code

The core interface already makes `T` and the projection specification visible. Preserve that clarity. Convenience defaults can exist for interactive use, but package internals and reusable examples should pass numerical policy explicitly.

The existing `with_default_*` functions should remain consumption combinators only: they call a function with the current process-global value and neither mutate nor restore state. If temporary overrides are ever introduced, make them task-local and give them a distinct interface rather than overloading `with_default_*` with a second meaning.

4. Extend the performance evidence before optimizing

The published benchmark report shows that common hot paths are already excellent. I would not spend the next cycle shaving another fraction of a nanosecond from decode or deterministic projection. Instead, extend the report to cover the implementation's actual architectural range:

- at least one representative format from each of the three carrier rungs;
- $K > 8$ scalar operations, array kernels, table-policy decisions, and block operations;
- cold compilation/first-call time separately from warm execution;
- one, two, and several Julia threads for advertised threaded paths;
- deterministic and stochastic calls with explicit RNGs;
- commit SHA, dirty state, CPU, Julia build, thread count, and benchmark-suite revision.

The fast and full reports should share case identifiers so differing results can be traced to workload or environment rather than compared informally.

Optimize only the reported allocation centers

After broadening the evidence, profile these paths in order:

1. Packed-vector `vmap`: eliminate per-tile views or transient scratch objects and target a small, size-independent allocation count.
2. Cold binary table construction: write result code units directly where safe, reuse construction buffers, and inspect closure/iterator boxing. Preserve the rule that tables are built through scalar semantics.

3. `BlockAdd` and `ConvertToBlockMaxAbsFinite`: inspect enclosure, tuple, and temporary carrier creation; introduce in-place internal kernels only when they preserve the public exact-project contract.
4. Wide-carrier first-call latency: precompile representative operation families only if report evidence shows meaningful user cost. Avoid combinatorial precompilation of all 504 formats.

Every optimization should be guarded by semantic differential checks and an allocation/performance case. The table/kernel seam should remain intact: execution strategy may change, numerical meaning may not.

5. Improve source locality without flattening deep modules

Keep source comments close to current invariants: why one projection is sufficient, why a carrier rung is valid, why a sticky bit preserves the rounding decision, and why a cache size is safe. Move chronological M-number notes, past implementation alternatives, and long postmortems into dated design records linked from the relevant source.

Do not split `oracle.jl` or `dyadic.jl` merely because they are large. Split only if a candidate module has its own durable interface and hides a meaningful implementation—for example, exact dyadic primitives separated from operation-specific enclosure construction. A collection of one-function forwarding files would reduce depth and worsen navigation.

Make the verification gates data-driven. A small manifest describing each gate, tier, exhaustive/sampled status, expected artifact, and approximate cost would improve both test-runner locality and documentation generation.

Proposed delivery sequence

Phase A: truth repair

- Fix or deprecate inert RNG/Rbits controls.
- Generate the conformance version and record exact draft identity.
- Unify the rounding taxonomy.
- Correct known documentation contradictions.

Exit condition: every exported setting has a behavioral check, and every conformance identity comes from one source of truth.

Phase B: automation and hygiene

- Add tiered CI and release metadata.
- Generate structured documentation facts.
- Remove generated PDF build intermediates from version control.
- Establish release tags and a checklist.

Exit condition: a clean checkout can reproduce the published documentation and conformance artifacts through documented commands.

Phase C: interface refinement

- Add explicit checked code-point conversion.
- Deprecate unchecked or mutable implementation exports.
- Clarify namespace organization, extension policy, and state semantics.

Exit condition: the common interface is smaller and safer while existing callers have a documented migration path.

Phase D: measured performance work

- Expand benchmark coverage to all carrier rungs and reproducibility metadata.
- Optimize packed mapping, cold table construction, and block allocations.

- Add only evidence-justified precompilation.

Exit condition: published results substantiate the full supported format range, and targeted allocation reductions do not change numerical semantics.

Changes I would deliberately avoid

- Do not rewrite the exact oracle or projection engine without a demonstrated correctness or performance failure.
- Do not add implicit cross-format promotion; explicit conversion is part of the numerical contract.
- Do not collapse rounding modes into runtime symbols or booleans; their type-level representation is what permits zero-overhead specialization.
- Do not materialize tables without respecting the existing bit-budget policy.
- Do not make approximation the silent default; keep it opt-in and measured.
- Do not fragment deep numerical modules into shallow wrappers for cosmetic file-size reduction.
- Do not optimize already sub-10-nanosecond hot paths before addressing report coverage and allocation-heavy paths.

Desired end state

SmallFloats.jl should retain its present mathematical identity: a precise, explicit implementation of small floating-point arithmetic with a single projection seam. The improved package would add operational exactness around that core—truthful controls, generated conformance facts, reproducible reports, automated gates, safer representation adapters, and a public interface whose supported extension points are unmistakable. That work would turn an excellent numerical implementation into a dependable package release process without sacrificing its strongest design choices.